Construir um modelo de Inteligência Artificial capaz de classificar imagens considerando 10 categorias: ['avião', 'automóvel', 'pássaro', 'gato', 'cervo', 'cachorro', 'sapo', 'cavalo', 'navio', 'caminhão']. Dada uma nova imagem de uma dessas categorias o modelo deve ser capaz de classificar e indicar o que é a imagem. Lembrando: uma imagem é uma matriz de pixels!

Double-cliquez (ou appuyez sur Entrée) pour modifier

```
# Versão da Linguagem Python
from platform import python_version
print('Versão do Python neste Notebook:', python_version())
```

```
Versão do Python neste Notebook: 3.12.13
```

```
# Instala o TF
#!pip install -q tensorflow==2.12
!pip install -q tensorflow
```

```
# Silencia mensagens do TF
%env TF_CPP_MIN_LOG_LEVEL=3
```

```
env: TF_CPP_MIN_LOG_LEVEL=3
```

Double-cliquez (ou appuyez sur Entrée) pour modifier

```
# Imports
import tensorflow as tf
import tensorflow_datasets as tfds
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.metrics import confusion_matrix
from PIL import Image
import os
```

Carregando os dados - matéria prima

https://www.cs.toronto.edu/~kriz/cifar.html

```
# Carregando o STL-10
(ds_train, ds_test), ds_info = tfds.load(
    'stl10',
    split=['train', 'test'],
    as_supervised=True,
    with_info=True
)
```

```
WARNING:absl:Variant folder /root/tensorflow_datasets/stl10/1.0.0 has no dataset_info.json
Downloading and preparing dataset Unknown size (download: Unknown size, generated: Unknown size
Dl Completed...: 100%    1/1 [02:34<00:00, 94.84s/ url]

Dl Size...: 100%    2518/2518 [02:34<00:00, 28.09 MiB/s]

Extraction completed...: 100%    7/7 [02:34<00:00, 154.67s/ file]

Dataset stl10 downloaded and prepared to /root/tensorflow_datasets/stl10/1.0.0. Subsequent call
```

```
# Nomes das classes oficiais do STL-10
nomes_classes = ['avião', 'pássaro', 'carro', 'gato', 'veado', 'cachorro', 'cavalo', 'macaco',
```

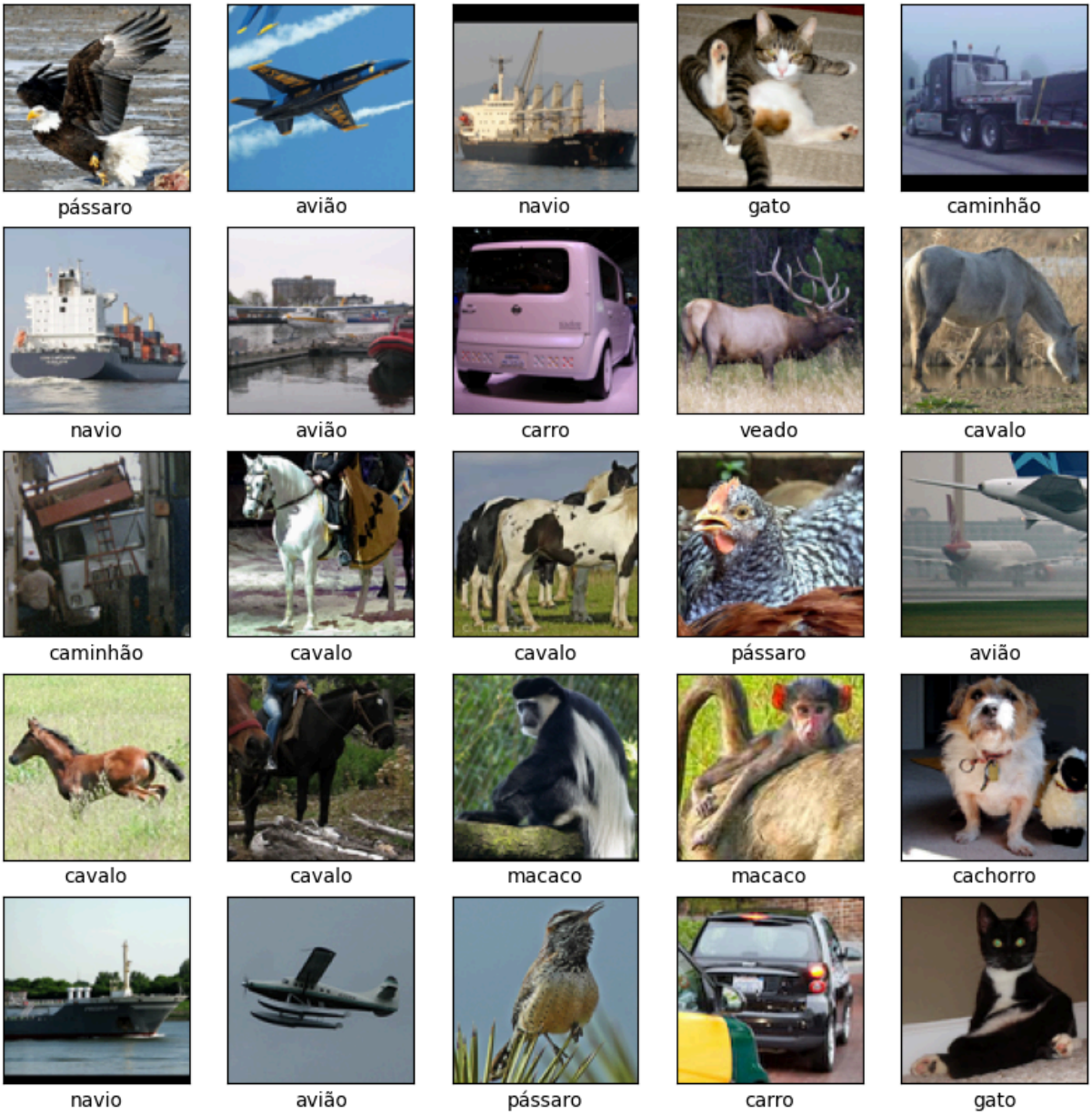
```
# Normaliza os valores dos pixels para que os dados fiquem na mesma escala
def preprocess(image, label):
    image = tf.cast(image, tf.float32) / 255.0
    return image, label
```

```
# Função para exibir as imagens
def visualiza_amostras(dataset, nomes_classes):
    plt.figure(figsize=(10, 10))
    # Pegamos um "batch" (lote) de imagens para mostrar
    for imagens, labels in dataset.take(1):
        for i in range(25):
            plt.subplot(5, 5, i + 1)
            plt.xticks([])
            plt.yticks([])
            plt.grid(False)
            plt.imshow(imagens[i])
            # No STL-10, o label vem como um número escalar
            plt.xlabel(nomes_classes[labels[i].numpy()])
    plt.show()
```

Gemini

```
BATCH_SIZE = 32

train_data = ds_train.map(preprocess).batch(BATCH_SIZE)
test_data = ds_test.map(preprocess).batch(BATCH_SIZE)
```

```
# Executa a função
#Imagens de trino
visualiza_amostras(train_data, nomes_classes)
```




```
#AGR que tem tudo, vai criar o modelo
```

Construção do Modelo

```
#Tem 3 camadas por que é RGB# Arquitetura do Modelo -> Feature Learning

modelo_lia = models.Sequential([
    # 1. Feature Learning (O que você já tinha)
    layers.Input(shape=(96, 96, 3)),

    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
```

```
    # 2. Classification
    layers.Flatten(), # Transforma os (10, 10, 128) em um único vetor
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2), # Segurança contra overfit
    layers.Dense(10, activation='softmax') # A saída de 10 classes
])
```

Commencez à coder ou à générer avec l'IA.

```
#Vamo ver se a nossa aquitetura ta ok
```

```
# Sumário do modelo
modelo_lia.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 94, 94, 32)	896
max_pooling2d_12 (MaxPooling2D)	(None, 47, 47, 32)	0
conv2d_13 (Conv2D)	(None, 45, 45, 64)	18,496
max_pooling2d_13 (MaxPooling2D)	(None, 22, 22, 64)	0
conv2d_14 (Conv2D)	(None, 20, 20, 128)	73,856
max_pooling2d_14 (MaxPooling2D)	(None, 10, 10, 128)	0
flatten_3 (Flatten)	(None, 12800)	0
dense_6 (Dense)	(None, 128)	1,638,528
dropout_3 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 10)	1,290

Total params: 1,733,066 (6.61 MB)
Trainable params: 1,733,066 (6.61 MB)
Non-trainable params: 0 (0.00 B)

```
#O deepseek tem 700 Bilhoes de parâmetros
#Vai usar a métrica de acurácia para ver o sucesso do modelo
#Não existe modelo com 100% e nem com 0%
#se ficar muito perto de 100%, desconfiar
#O olo humano da 70# em cancewr de pulmao, as IAs tem 90%
# Compilação do modelo
modelo_lia.compile(optimizer = 'adam',
                   loss = 'sparse_categorical_crossentropy',
                   metrics = ['accuracy'])

# adam - é um algoritmo de aprendizagem tipo backpropagation!
# loss - função de erro. Isso se resume a uma otimização função matemática!
# metrics - medir o sucesso!
```

```
#epochs é as temporadas, quantas vezes tem que passar por todo o dataset pra aperfeiçoar
#abaixo de 100 é brincadeira de criança
```

```
#Quanto mais ele roda, mais aprende, mais tem um momento em que ele começa a desaprender
#Ele desaptrnde por que pode começar a viciar e a tender
#Pode ser que o dataset seja desbalanceado, tipo 1000 de
#Overfit
```

```
%%time
history = modelo_lia.fit(train_data, epochs=100, validation_data=test_data)


modelo_lia.save('modelo_stl10_treinado.h5')
print("Modelo salvo com sucesso!")
```

Commencez à coder ou à générer avec l'IA.

Avaliação do modelo

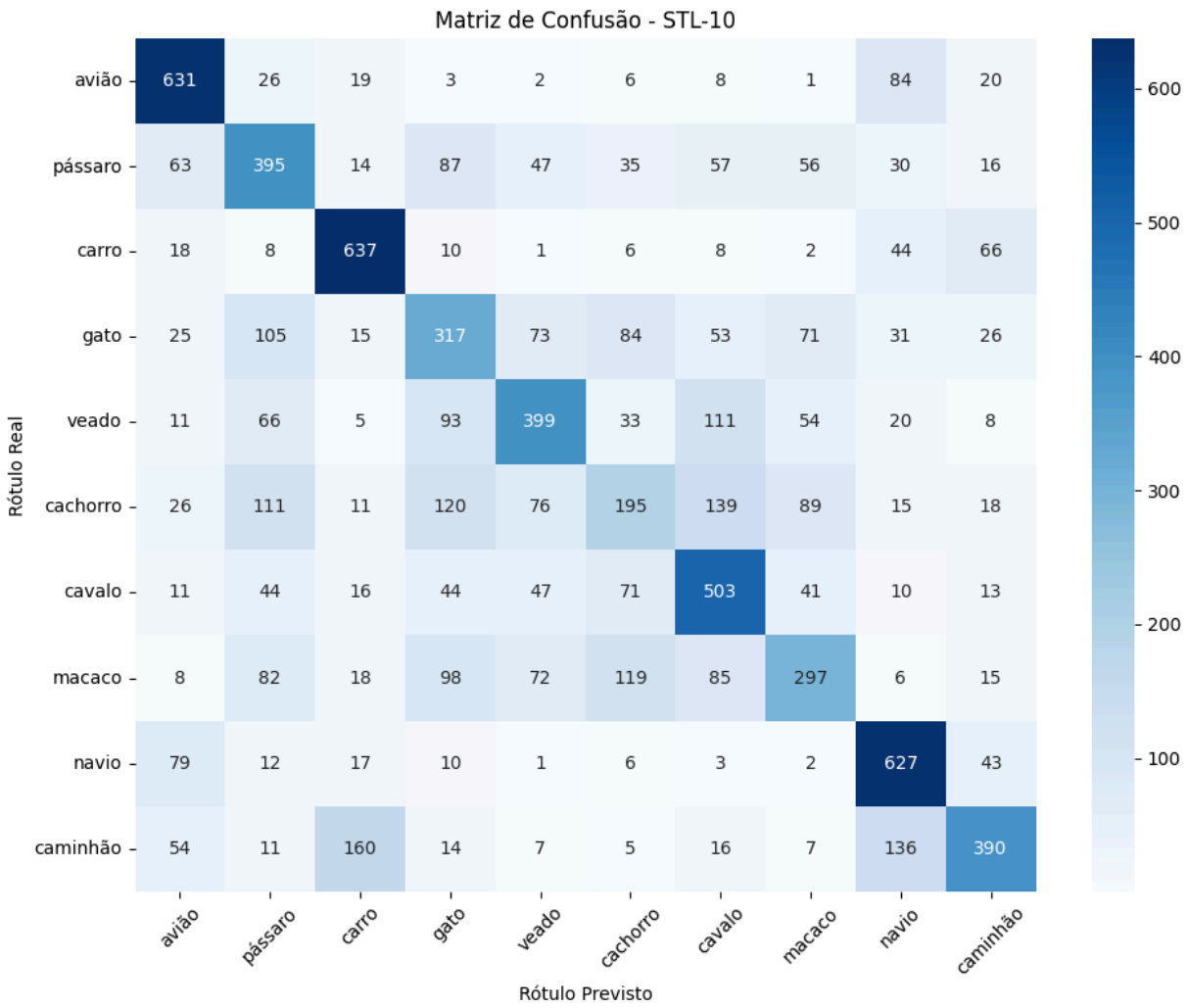
```
# Avaliação do modelo
erro_teste, acc_teste = modelo_lia.evaluate(test_data, verbose=2)

print(f'\nAcurácia no conjunto de teste: {acc_teste:.2%}')
```

```
250/250 - 36s - 144ms/step - accuracy: 0.5489 - loss: 4.8229

Acurácia no conjunto de teste: 54.89%
```

```
print('\nAcurácia com dados de Teste:', acc_teste)
```

```
Acurácia com dados de Teste: 0.5488749742507935
```

```
# --- Adaptação para Matriz de Confusão no STL-10 ---
from sklearn.metrics import confusion_matrix
import seaborn as sns

# 1. Coleta as previsões e os rótulos reais do pipeline de teste
y_true = []
y_pred_classes = []

# Iteramos sobre o dataset de teste para pegar as imagens e as labels reais
for imagens, labels in test_data:
    preds = modelo_lia.predict(imagens, verbose=0)
    y_pred_classes.extend(np.argmax(preds, axis=1))
    y_true.extend(labels.numpy())

# 2. Geração da matriz de confusão
cm = confusion_matrix(y_true, y_pred_classes)

# 3. Impressão Visual
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=nomes_classes,
            yticklabels=nomes_classes)

plt.title('Matriz de Confusão - STL-10')
plt.xlabel('Rótulo Previsto')
plt.ylabel('Rótulo Real')
plt.xticks(rotation=45)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```




```
# Carrega uma nova imagem
nova_imagem = Image.open("maine-coon0000.jpg.avif")
```

```
# Dimensões da imagem (em pixels)
nova_imagem.size

# Obtém largura e altura da imagem
largura = nova_imagem.width
altura = nova_imagem.height
```

```
print("A largura da imagem é: ", largura)
print("A altura da imagem é: ", altura)
```

```
A largura da imagem é:  1200
A altura da imagem é:  738
```

```
# Redimensiona para 96x96 pixels
nova_imagem = nova_imagem.resize((96, 96))
```

```
# Exibir a imagem
plt.figure(figsize = (1,1))
plt.imshow(nova_imagem)
plt.xticks([])
plt.yticks([])
plt.show()
```

```
# Converte a imagem para um array NumPy e normaliza
nova_imagem_array = np.array(nova_imagem) / 255.0
```

```
# Expande a dimensão do array para que ele tenha o formato (1, 32, 32, 3)
nova_imagem_array = np.expand_dims(nova_imagem_array, axis = 0)
```

```
# Previsões
previsoes = modelo_lia.predict(nova_imagem_array)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step
```

```
print(previsoes)
```

```
[[7.9892457e-09 8.8831475e-06 6.4806653e-14 9.9813509e-01 5.7389667e-07
  1.1281125e-03 1.0684692e-05 7.1072811e-04 1.1021153e-08 5.7994462e-06]]
```

```
# Obtém a classe com maior probabilidade e o nome da classe
classe_prevista = np.argmax(previsoes)
nome_classe_prevista = nomes_classes[classe_prevista]
```